IAPT Project: Interactive Augmented Reality Gallery

1. Introduction

When starting this project, the main idea was basically to figure out how we could make static things—like the murals on the hospital walls—interactive and a bit more fun, especially for kids who might be feeling stressed or bored. The goal was to build an Augmented Reality (AR) experience that didn't require anyone to download a heavy app from the App Store or Google Play. People usually don't want to download an app they're only going to use once. So, the solution had to be 100% web-based.

I wanted users to simply scan a QR code with their phone's native camera, click a link, and immediately see the artwork come to life. Sounds straightforward on paper, but getting web browsers to handle AR smoothly and securely turned out to be a massive learning curve. This report breaks down the entire journey of building this project, the technical choices I made, the headaches I ran into, and how I eventually solved them.

2. Context and Motivation

Hospitals are notoriously intimidating environments. The murals are already a great touch to make the space feel less clinical, but I figured we could push it a step further. If a kid is holding a parent's phone, pointing it at a mural and seeing a 3D animation or a video play right on top of the physical wall changes the dynamic completely. It becomes a game.

The strict constraint here was accessibility. If the barrier to entry is too high (like needing to create an account, download a 100MB app, or wait 5 minutes for assets to load), people just won't use it. WebAR was the only logical path forward. It runs right in Safari or Chrome, leveraging the device's camera.

3. Technology Stack Selection

Picking the right tools was actually one of the hardest parts of the project because the WebAR ecosystem is quite fragmented.

3.1 Frontend: HTML, CSS, and Vanilla JavaScript

I decided to stick to Vanilla JavaScript instead of jumping into heavy frameworks like React or Vue. Given that AR relies heavily on the device's GPU and memory, I wanted the browser to use as much of its resources as possible for rendering the camera feed and the 3D scene, rather than computing complex virtual DOM diffs. Plus, keeping the tech stack lightweight meant faster initial load times.

3.2 AR Engine: A-Frame and MindAR

For the actual augmented reality side of things, I went with A-Frame paired with MindAR. - **A-Frame:** It's an entity-component framework based on Three.js. It basically lets you write 3D scenes using HTML-like tags (like `<a-scene>` or `<a-camera>`). It made placing the video overlays super intuitive. - **MindAR:** This was the real game-changer. I initially looked into AR.js, but MindAR specifically specializes in image tracking (which is exactly what I needed to track the murals) and it handles multiple targets really well. It uses TensorFlow.js under the hood to extract feature points from the images and map them in real-time.

3.3 Backend: Python HTTPS Server

Because this project needs to be run locally for the assessment, I needed a way to serve the files to the mobile device. I wrote a custom Python script using the built-in `http.server` module. However, modern browsers have strict security policies: they flat out refuse to open the camera hardware unless the connection is secure (HTTPS). Since I was running this on a local IP network, getting a real SSL certificate wasn't possible. So, I coded the Python script to dynamically generate a self-signed SSL certificate using OpenSSL/cryptography on startup. It feels a bit hacky because the browser throws a "Not Secure" warning that the user has to bypass, but for a local development setup, it was the only viable workaround to get camera access.

4. System Architecture

The architecture is basically split into two main components: the "Connection Portal" (the desktop view) and the "AR Experience" (the mobile view).

1. **The Server Layer:** When `start_https_server.py` runs, it finds the computer's local IP address (e.g., 192.168.x.x), creates the SSL certs, and starts listening on port 8443.
2. **The Connection Portal:** The script automatically opens a web page on the host computer. This page has a JavaScript function that pulls the local IP and generates a QR code using `qrcodejs`.

3. **The Handshake:** The user scans this QR code with their phone. Since the phone and computer are on the same Wi-Fi, the phone connects directly to the Python server.
4. **The Client Rendering:** The phone downloads the `index.html` AR page. A-Frame initializes the WebGL canvas, MindAR requests camera permissions, and the tracking loop begins.

5. UI/UX Design Approach

Originally, I thought about making the UI very colorful and playful (think fish emojis, bubbles, ocean themes), but I realized that AR works best when the interface gets out of the way. If the screen is cluttered with UI elements, it distracts from the actual mural animation.

I pivoted to what I call a "Modern Museum" aesthetic. I used a very clean, minimalist light mode design. - **Typography:** I imported the 'Inter' font from Google Fonts. It's incredibly legible on small mobile screens. - **Visual Feedback:** I added a dynamic "Tracking Signal" bar. One issue with AR is that when it's scanning, the user doesn't know if the app is frozen or actually looking for an image. The moving signal bar acts as a psychological cue that the system is working. - **Onboarding Tips:** I implemented a rotating text box that gives users hints (e.g., "Ensure the area is well-lit", "Move closer"). This was crucial because people often hold their phones too far away or stand in dark corners where the AI can't detect the image features.

6. Major Technical Challenges

This project definitely didn't go smoothly on the first try. Here are the biggest roadblocks I hit and how I fixed them.

6.1 The Memory Crash Bottleneck

The absolute biggest issue I faced was getting the tracking data compiled. MindAR requires you to compile the target images into a `.mind` file. Initially, the 9 mural images were massive—around 5 to 6 megabytes each. When I fed these into the MindAR browser compiler, the browser's memory would just spike and silently crash. I spent hours thinking my code was broken. The fix was optimizing the source imagery. By resizing the images and compressing them down to around 200KB each, the compiler could easily process the feature points without overloading the browser's RAM. It taught me a hard lesson about asset management in web applications.

6.2 Browser Autoplay Policies

Another headache was audio. Browsers like Safari have strict rules that prevent videos with sound from playing automatically unless the user interacts with the screen first. When the AR engine detected a mural, my code would try to play the `.mp4` file. If the browser blocked it, the video would just freeze. To get around this, I programmed the video to start muted. If the system catches an autoplay rejection, it spawns a "Tap to Play" button on the screen, forcing the user to interact, which then unlocks the audio context.

6.3 Service Workers and Caching

Downloading the `targets.mind` file (which holds all the AI tracking data) and the 9 video files takes a lot of bandwidth. On a slow Wi-Fi network, the user would be staring at a loading screen for over a minute. I wrote a Service Worker (`sw.js`) to intercept network requests. The first time the user loads the app, it caches all the heavy assets. If they close the browser and come back later, or even if they lose internet connection but stay on the local network, the app loads almost instantaneously from the local cache.

7. Execution Instructions (For the Examiner)

If you are evaluating this project, please follow these exact steps to run it:

1. **Network Setup:** Ensure your computer and your testing mobile device are connected to the exact same Wi-Fi network.
2. **Start the Server:** Open your terminal, navigate to the `Name_Surname` folder, and run:
   ```
   python start_https_server.py
   ```
3. **The Portal:** Your desktop browser will automatically open the Connection Portal.
4. **Mobile Connection:** Open your mobile phone's camera and scan the QR code on your computer screen.
5. **Security Warning:** Because this uses a self-signed certificate for local testing, your phone will say the connection is not secure.
6. *On iOS Safari:* Tap "Show Details" -> "visit this website".
7. *On Android Chrome:* Tap "Advanced" -> "Proceed to [IP] (unsafe)".
8. **Experience AR:** Click "Initialize Camera", allow permissions, wait for the loading bar to finish, and point your phone at any of the 9 mural images.

8. Conclusion

Wrapping things up, this project was a huge learning experience in bridging the gap between web development and computer vision. Dealing with local HTTPS servers, raw

WebGL performance, and the quirks of mobile browsers was frustrating at times, but seeing the physical images actually come to life on the screen makes it totally worth it. The final application is robust, the tracking is sticky and reliable, and the clean museum UI makes it feel like a polished, professional product. I'm really proud of how the caching strategy turned out, as it makes the UX significantly smoother after that initial hit.

I believe this completely fulfills the requirements of the IAPT project, providing an interactive, accessible, and engaging tool that could genuinely be used in a real-world scenario.